

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN – CƠ – TIN HỌC

SẮP XẾP SỰ KIỆN THỜI GIAN
TEMPORAL EVENT ORDERING UNDER
TEMPORAL MASKING

Nhóm sinh viên thực hiện:

Phạm Hoàng Huy – K68A3
Nguyễn Trần Phúc Châu – K68A3
Phạm Dương Hoàng – K68A3

Môn học: Phương pháp nghiên cứu khoa học

GIẢNG VIÊN HƯỚNG DẪN:

TS. Hà Mỹ Linh

HÀ NỘI – 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN – CƠ – TIN HỌC

SẮP XẾP SỰ KIỆN THỜI GIAN
TEMPORAL EVENT ORDERING UNDER
TEMPORAL MASKING

Nhóm sinh viên thực hiện:

Phạm Hoàng Huy – K68A3
Nguyễn Trần Phúc Châu – K68A3
Phạm Dương Hoàng – K68A3

Môn học: Phương pháp nghiên cứu khoa học

GIẢNG VIÊN HƯỚNG DẪN:

TS. Hà Mỹ Linh

HÀ NỘI – 2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan báo cáo nghiên cứu này là công trình khoa học do nhóm thực hiện một cách độc lập và nghiêm túc, dưới sự hướng dẫn khoa học của TS. Hà Mỹ Linh. Các số liệu, kết quả thực nghiệm và nội dung được trình bày trong báo cáo là hoàn toàn trung thực, khách quan và chưa từng được công bố ở bất kỳ tài liệu hay công trình nào khác.

Toàn bộ các thông tin, tài liệu tham khảo sử dụng trong báo cáo đều được trích dẫn, ghi nguồn đầy đủ và hợp pháp. Mã nguồn và sản phẩm công nghệ đi kèm được phát triển dựa trên năng lực của nhóm và được minh bạch hóa trong tài liệu này.

Chúng tôi xin hoàn toàn chịu trách nhiệm trước Nhà trường, Khoa và Hội đồng về tính chính xác và trung thực của lời cam đoan trên.

Hà Nội, tháng 5 năm 2026

Nhóm nghiên cứu

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài, nhóm nghiên cứu đã nhận được sự quan tâm, hướng dẫn và hỗ trợ quý báu từ thầy cô, nhà trường và cộng đồng học thuật. Chúng tôi xin bày tỏ lòng biết ơn chân thành đến tất cả những cá nhân và tập thể đã đồng hành, tạo điều kiện để nhóm hoàn thành báo cáo này.

Trước hết, chúng tôi xin gửi lời cảm ơn sâu sắc đến **TS. Hà Mỹ Linh**, giảng viên hướng dẫn đề tài, đã tận tình định hướng, góp ý chuyên môn và hỗ trợ nhóm trong suốt quá trình nghiên cứu. Những nhận xét và chỉ dẫn của cô là cơ sở quan trọng giúp nhóm hoàn thiện phương pháp tiếp cận, nội dung thực nghiệm và cách trình bày báo cáo.

Chúng tôi cũng xin trân trọng cảm ơn **Khoa Khoa học máy tính và thông tin** đã tạo môi trường học tập, nghiên cứu thuận lợi, cung cấp nền tảng kiến thức và điều kiện cần thiết để nhóm triển khai đề tài. Sự hỗ trợ từ Khoa là động lực quan trọng giúp nhóm tiếp cận bài toán một cách nghiêm túc và có hệ thống.

Bên cạnh đó, nhóm xin cảm ơn cộng đồng **Wikipedia tiếng Việt** đã xây dựng và duy trì nguồn tri thức mở phong phú, đóng vai trò là nguồn dữ liệu chính cho nghiên cứu. Chúng tôi cũng biết ơn các tác giả của những công trình khoa học, thư viện mã nguồn mở và tài liệu kỹ thuật đã được tham khảo trong quá trình thực hiện đề tài.

Do hạn chế về thời gian, kinh nghiệm và nguồn lực, báo cáo khó tránh khỏi những thiếu sót. Nhóm nghiên cứu kính mong nhận được ý kiến đóng góp từ quý thầy cô và bạn đọc để tiếp tục hoàn thiện hướng nghiên cứu trong tương lai.

TÓM TẮT

Tên đề tài: Sắp xếp sự kiện thời gian trong văn bản tiếng Việt khi che mất yếu tố thời gian (*Temporal Event Ordering under Temporal Masking*)

Từ khóa: Xử lý ngôn ngữ tự nhiên, suy luận thời gian, sắp xếp sự kiện, tiếng Việt, PhoBERT, Wikipedia

Nghiên cứu này tập trung vào bài toán sắp xếp sự kiện thời gian trong văn bản tiếng Việt, đặc biệt trong bối cảnh các mốc thời gian tường minh bị che khuất một phần hoặc hoàn toàn. Đây là bài toán đòi hỏi mô hình không chỉ nhận diện ngày, tháng, năm trong câu, mà còn phải khai thác ngữ nghĩa, quan hệ nguyên nhân – kết quả, luồng tự sự và kiến thức nền để khôi phục trật tự hợp lý của các sự kiện.

Báo cáo xây dựng bộ dữ liệu TEO-TM tiếng Việt từ Wikipedia tiếng Việt với quy mô **hơn 300.000 sự kiện** thuộc **hơn 20.000 nhân vật**. Quy trình thu thập và xử lý dữ liệu được thiết kế thành pipeline tự động gồm năm giai đoạn: thu thập URL nhân vật, trích xuất sự kiện từ nội dung bài viết, lọc dữ liệu qua nhiều lớp kiểm tra, xử lý theo các batch song song và hợp nhất dữ liệu cuối cùng. Trên cơ sở bộ dữ liệu này, nghiên cứu đề xuất mô hình **PhoBERT Pairwise Ranking**, huấn luyện theo từng cặp sự kiện và sử dụng cơ chế tổng hợp thứ hạng **Borda Count** để suy ra thứ tự toàn chuỗi.

Thực nghiệm được tiến hành trên ba thiết lập: giữ nguyên thời gian (AT), che một phần thời gian (PM) và che toàn bộ thời gian (MT). Kết quả cho thấy PhoBERT Pairwise đạt hiệu quả cao nhất trong các mô hình được so sánh, với **Kendall Tau 0.8756** và **Spearman's Rho 0.9331** ở thiết lập AT; đồng thời vẫn duy trì khả năng suy luận tương đối tốt khi toàn bộ mốc thời gian bị che với **Kendall Tau 0.4795**. So sánh với các mô hình ngôn ngữ lớn theo hướng zero-shot prompting như DeepSeek-7B và Qwen2.5-14B cho thấy mô hình fine-tune chuyên biệt phù hợp hơn với bài toán TEO-TM tiếng Việt. Các kết quả cũng chỉ ra rằng việc khôi phục chính xác toàn bộ chuỗi sự kiện dài khi thiếu mốc thời gian vẫn là một thách thức mở, cần tiếp tục cải thiện bằng tri thức nền và các kiến trúc mô hình nâng cao.

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
NLP	Natural Language Processing – Xử lý ngôn ngữ tự nhiên
TEO	Temporal Event Ordering – Sắp xếp sự kiện thời gian
TEO-TM	Temporal Event Ordering under Temporal Masking – Sắp xếp sự kiện thời gian khi che mốc thời gian
AT	Absolute Time – Thiết lập giữ nguyên mốc thời gian
PM	Partially Masked Time – Thiết lập che một phần mốc thời gian
MT	Masked Time – Thiết lập che toàn bộ mốc thời gian
BERT	Bidirectional Encoder Representations from Transformers
PhoBERT	Vietnamese BERT pretrained model – Mô hình BERT tiền huấn luyện cho tiếng Việt
mBERT	Multilingual BERT – Mô hình BERT đa ngôn ngữ
LLM	Large Language Model – Mô hình ngôn ngữ lớn
GPT	Generative Pre-trained Transformer
NLI	Natural Language Inference – Suy luận ngôn ngữ tự nhiên
NER	Named Entity Recognition – Nhận dạng thực thể có tên
POS	Part-of-Speech – Gán nhãn từ loại
SVM	Support Vector Machine – Máy vectơ hỗ trợ
TF-IDF	Term Frequency – Inverse Document Frequency
F1	F1-score – Độ đo hài hòa giữa Precision và Recall
API	Application Programming Interface – Giao diện lập trình ứng dụng

Từ viết tắt	Giải thích
URL	Uniform Resource Locator – Địa chỉ tài nguyên trên web
CSV	Comma-Separated Values – Định dạng dữ liệu phân cách bằng dấu phẩy
UTF-8	Unicode Transformation Format – 8 bit
BOM	Byte Order Mark – Dấu nhận diện mã hóa ở đầu tệp
BFS	Breadth-First Search – Tìm kiếm theo chiều rộng
regex	Regular Expression – Biểu thức chính quy
GNN	Graph Neural Network – Mạng nơ-ron đồ thị
QA	Question Answering – Hỏi đáp tự động

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Danh mục từ viết tắt	iv
Danh mục hình ảnh	ix
Danh mục bảng biểu	x
Mở đầu	1
1 Mô tả bài toán	3
1.1 Thông tin thời gian trong văn bản	3
1.1.1 Định nghĩa	3
1.1.2 Vai trò của thông tin thời gian	4
1.1.3 Phân loại thông tin thời gian trong tiếng Việt	4
1.2 Bài toán sắp xếp sự kiện thời gian khi che mốc thời gian	5
1.2.1 Động cơ của temporal masking	5
1.2.2 Phát biểu bài toán chính thức	5
1.2.3 Các thiết lập đánh giá trong báo cáo	6
1.2.4 Các thách thức chính	6
1.3 Các nghiên cứu liên quan	7
1.3.1 Các nghiên cứu trên thế giới	7
1.3.2 Các nghiên cứu trong nước	8
1.4 Mục tiêu đề tài	9
1.4.1 Mục tiêu 1: Xây dựng bộ dữ liệu	9
1.4.2 Mục tiêu 2: Xây dựng và đánh giá mô hình	10

1.5	Phạm vi nghiên cứu	10
1.6	Tổng kết chương	10
2	Xây dựng bộ dữ liệu sắp xếp sự kiện thời gian trong tiếng Việt	12
2.1	Xây dựng dữ liệu thủ công	12
2.1.1	Giai đoạn 1: Thu thập URL Wikipedia (data_downloading.py)	12
2.1.2	Giai đoạn 2: Xử lý URL (data_processing.py)	16
2.1.3	Giai đoạn 3: Trích xuất sự kiện (data_extracting.py)	16
2.1.4	Giai đoạn 4: Lọc và làm sạch (data_filtering.py)	18
2.1.5	Giai đoạn 5: Gộp dữ liệu (data_merging.py)	21
2.2	Thống kê bộ dữ liệu	22
2.3	Ngoại lệ bộ dữ liệu	23
2.4	Tổng kết chương	23
3	Xây dựng mô hình và Thuật toán suy luận	25
3.1	Tiền xử lý và định dạng dữ liệu đầu vào	25
3.1.1	Khung suy luận ngôn ngữ tự nhiên (NLI Framing)	25
3.1.2	Lấy mẫu theo cửa sổ trượt (Window Sampling)	26
3.2	Kiến trúc mô hình và Chiến lược học tập	26
3.2.1	Tinh chỉnh PhoBERT với lớp phân loại vô hướng	26
3.2.2	Tối ưu hóa với Margin Ranking Loss	27
3.3	Thuật toán Borda Count khôi phục chuỗi thời gian	27
3.4	Tổng kết chương	28
4	Đánh giá mô hình và So sánh thực nghiệm	29
4.1	Phương pháp đánh giá và Các hệ đo lường	29
4.2	Kết quả đánh giá trên tập xác thực (Validation Metrics)	30
4.3	Đánh giá chéo trên tập kiểm thử thu nhỏ (Cross-Evaluation 3x3)	30
4.3.1	Nhận định dựa trên các hệ số đo lường	31
4.4	So sánh đối chiếu với các mô hình ngôn ngữ lớn (LLMs)	32
4.4.1	Đánh giá năng lực sinh ngôn ngữ so với xếp hạng cặp	32
4.5	Tổng kết chương	33
	Kết luận	34
A	Cấu trúc thư mục dự án	36

B	Danh sách danh mục Wikipedia sử dụng	37
C	Môi trường thực nghiệm	38
	Tài liệu tham khảo	39

Danh sách hình vẽ

2.1	Pipeline xây dựng bộ dữ liệu TEO-TM tiếng Việt	12
2.2	Pipeline lọc dữ liệu sáu lớp trong <code>data_filtering.py</code>	18

Danh sách bảng

1.1	Phân loại thông tin thời gian trong văn bản tiếng Việt	4
2.1	Thống kê quá trình thu thập URL	16
2.2	Thống kê tổng quan bộ dữ liệu TEO-TM tiếng Việt	22
2.3	Phân bố loại nhân vật trong bộ dữ liệu	23
4.1	Kết quả tốt nhất ghi nhận trong quá trình huấn luyện (In-domain Validation)	30
4.2	Ma trận thực nghiệm đánh giá chéo (Cross-Evaluation) trên 250 nhân vật	31
4.3	So sánh đa chiều hiệu suất sắp xếp sự kiện giữa PhoBERT Pairwise và LLMs	32
B.1	Các danh mục Wikipedia tiếng Việt làm seed crawl	37
C.1	Cấu hình phần cứng và phần mềm thực nghiệm	38

MỞ ĐẦU

Lý do chọn đề tài

Trong thời đại bùng nổ thông tin, văn bản tiếng Việt xuất hiện ngày càng nhiều trên các nền tảng trực tuyến như Wikipedia, báo điện tử, mạng xã hội, và các hệ thống lưu trữ tri thức. Một trong những thách thức quan trọng trong Xử lý Ngôn ngữ Tự nhiên (NLP) là hiểu và tổ chức *thông tin thời gian* – tức là xác định thứ tự xảy ra của các sự kiện được mô tả trong văn bản.

Bài toán sắp xếp sự kiện thời gian (*Temporal Event Ordering* – TEO) đã được nghiên cứu rộng rãi trong tiếng Anh và một số ngôn ngữ châu Âu, song đối với tiếng Việt, đây vẫn là lĩnh vực còn rất mới mẻ và thiếu tài nguyên. Đặc biệt, khi các mốc thời gian tường minh (như năm, tháng, ngày) bị che đi hoặc không có trong văn bản – trường hợp phổ biến trong văn bản tường thuật, tiểu sử và lịch sử – bài toán trở nên phức tạp hơn nhiều và đòi hỏi mô hình phải dựa vào ngữ nghĩa, ngữ cảnh và kiến thức nền để suy luận thứ tự.

Phạm vi và đối tượng nghiên cứu

Nghiên cứu tập trung vào văn bản tiếng Việt mô tả tiểu sử nhân vật, thu thập từ Wikipedia tiếng Việt. Đối tượng nghiên cứu bao gồm:

- Các câu văn mô tả sự kiện trong cuộc đời nhân vật lịch sử và đương đại.
- Các mô hình NLP và học sâu được áp dụng cho tiếng Việt.
- Quy trình xây dựng bộ dữ liệu NLP chất lượng cao từ nguồn mở.

Phương pháp nghiên cứu

Nghiên cứu sử dụng phương pháp thực nghiệm (empirical), bao gồm: thu thập và xử lý dữ liệu tự động bằng Python; xây dựng và đánh giá mô hình theo phương pháp học có giám sát; và phân tích lỗi định tính để hiểu sâu hạn chế của từng phương pháp.

Đóng góp chính

1. **Bộ dữ liệu TEO-TM tiếng Việt đầu tiên:** Hơn 300.000 sự kiện từ hơn 20.000 nhân vật, được thu thập và lọc tự động qua pipeline 5 giai đoạn.
2. **Benchmark toàn diện:** So sánh bốn nhóm phương pháp trên cùng điều kiện thực nghiệm, cung cấp cơ sở tham chiếu cho nghiên cứu tương lai.
3. **Phân tích lỗi định tính:** Xác định bốn nhóm lỗi chính, định hướng cải tiến.

Cấu trúc báo cáo

Báo cáo được tổ chức thành các chương như sau:

- **Chương 1 – Mô tả bài toán:** Định nghĩa bài toán TEO-TM, tổng quan nghiên cứu liên quan trong và ngoài nước, mục tiêu đề tài.
- **Chương 2 – Xây dựng bộ dữ liệu:** Chi tiết pipeline tự động thu thập và làm sạch dữ liệu từ Wikipedia tiếng Việt, thống kê và các ngoại lệ.
- **Chương 3 – Xây dựng mô hình:** Bốn nhóm phương pháp thực nghiệm, kết quả so sánh, và phân tích lỗi.
- **Chương 4 – Kết luận:** Thành tích đạt được và hướng phát triển tương lai.

CHƯƠNG 1

MÔ TẢ BÀI TOÁN

1.1 Thông tin thời gian trong văn bản

1.1.1 Định nghĩa

Thông tin thời gian (*temporal information*) trong văn bản là tập hợp các biểu thức, từ ngữ, cấu trúc cú pháp và quan hệ ngữ nghĩa dùng để mô tả thời điểm, khoảng thời gian, trình tự hoặc mối liên hệ thời gian giữa các sự kiện. Trong các văn bản tường thuật, tiểu sử, lịch sử hoặc tin tức, thông tin thời gian đóng vai trò quan trọng vì nó giúp người đọc hiểu được diễn biến của sự việc: sự kiện nào xảy ra trước, sự kiện nào xảy ra sau, sự kiện nào là nguyên nhân hoặc hệ quả của sự kiện khác.

Theo chuẩn TimeML [1], thông tin thời gian thường được mô hình hóa thông qua ba thành phần chính:

- **Biểu thức thời gian** (TIMEX3): các cụm từ chỉ thời điểm, khoảng thời gian hoặc tần suất, ví dụ “năm 1945”, “tháng 8 năm 2020”, “trong ba năm”.
- **Sự kiện** (EVENT): các hành động, trạng thái hoặc biến cố có thể được định vị trên trục thời gian, ví dụ “sinh ra”, “tốt nghiệp”, “được bổ nhiệm”, “qua đời”.
- **Quan hệ thời gian** (TLINK): liên kết mô tả quan hệ giữa hai sự kiện, hoặc giữa sự kiện và biểu thức thời gian, chẳng hạn BEFORE, AFTER, INCLUDES hoặc SIMULTANEOUS.

Trong phạm vi báo cáo này, thành phần được quan tâm nhất là quan hệ thứ tự giữa các sự kiện. Thay vì chỉ nhận diện xem trong câu có mốc thời gian hay không, nghiên cứu hướng đến việc xác định trật tự tương đối của các sự kiện khi mốc thời gian bị che đi. Điều này khiến bài toán gần hơn với năng lực đọc hiểu và suy luận ngữ cảnh của mô hình.

1.1.2 Vai trò của thông tin thời gian

Việc xử lý đúng thông tin thời gian có ý nghĩa quan trọng trong nhiều ứng dụng của xử lý ngôn ngữ tự nhiên. Trong hệ thống hỏi đáp, mô hình cần biết sự kiện nào xảy ra trước để trả lời các câu hỏi dạng “trước khi trở thành tổng thống, nhân vật này từng làm gì?”. Trong tóm tắt văn bản, các câu sự kiện cần được sắp xếp theo trình tự hợp lý để tạo ra bản tóm tắt dễ hiểu. Trong xây dựng dòng thời gian tiểu sử, hệ thống phải gom các sự kiện rời rạc thành một chuỗi phát triển theo thời gian.

Đối với tiếng Việt, bài toán này càng đáng chú ý vì văn bản tự nhiên thường sử dụng nhiều cách diễn đạt linh hoạt. Một sự kiện có thể được gắn với mốc năm rõ ràng, nhưng cũng có thể chỉ được liên kết bằng các từ như “sau đó”, “trước khi”, “khi còn”, hoặc hoàn toàn không có dấu hiệu thời gian tường minh. Vì vậy, nếu chỉ dựa vào biểu thức chính quy để trích xuất ngày tháng, hệ thống sẽ bỏ sót nhiều quan hệ thời gian quan trọng.

1.1.3 Phân loại thông tin thời gian trong tiếng Việt

Trong tiếng Việt, thông tin thời gian có thể được biểu hiện ở nhiều mức độ khác nhau. Báo cáo chia thành ba nhóm chính như trong Bảng 1.1. Cách phân loại này giúp làm rõ độ khó tăng dần của bài toán: từ nhận diện mốc thời gian trực tiếp đến suy luận thứ tự dựa trên ngữ cảnh.

Bảng 1.1. Phân loại thông tin thời gian trong văn bản tiếng Việt

Loại	Ví dụ	Độ khó
Tường minh	“Năm 1963, ông trở thành tư lệnh SANG.”	Dễ
Tương đối	“Sau khi quốc vương qua đời, ông lên ngôi.”	Trung bình
Ẩn	“Ông tham gia kháng chiến, sau đó được bầu làm chủ tịch.”	Khó

Biểu thức tường minh là các mốc thời gian được nêu trực tiếp trong câu, ví dụ “ngày 2 tháng 9 năm 1945”, “năm 2005”, “thế kỷ XIX”. Nhóm này thường có thể nhận diện bằng biểu thức chính quy hoặc các bộ nhận diện thực thể thời gian. Tuy nhiên, nhận diện được mốc thời gian chưa đồng nghĩa với hiểu được toàn bộ quan hệ giữa các sự kiện, vì một câu có thể chứa nhiều sự kiện hoặc nhiều mốc thời gian khác nhau.

Biểu thức tương đối không nêu mốc tuyệt đối mà mô tả quan hệ giữa các sự kiện thông qua các từ nối như “sau đó”, “trước đó”, “khi”, “trong thời gian”, “hai năm sau”. Để xử lý nhóm này, mô hình cần xác định sự kiện làm mốc tham chiếu và hiểu vai trò của từ nối trong câu. Đây là mức khó hơn vì quan hệ thời gian phụ thuộc vào ngữ cảnh xung quanh.

Thông tin thời gian ẩn là trường hợp câu không chứa biểu thức thời gian rõ ràng nhưng vẫn gợi ý thứ tự thông qua quan hệ nhân quả, quy luật đời người, tri thức lịch sử hoặc cấu trúc tường thuật. Ví dụ, trong tiểu sử nhân vật, “tốt nghiệp đại học” thường xảy ra trước “được bổ nhiệm làm giảng viên”, dù câu không nêu năm. Đây là loại thông tin khó nhất và là trọng tâm của nghiên cứu, vì nó buộc mô hình phải suy luận thay vì chỉ đọc trực tiếp mốc thời gian.

1.2 Bài toán sắp xếp sự kiện thời gian khi che mốc thời gian

1.2.1 Động cơ của temporal masking

Trong nhiều văn bản thực tế, mốc thời gian có thể bị thiếu, không đầy đủ hoặc không được biểu diễn theo định dạng thống nhất. Nếu mô hình chỉ học cách so sánh các con số năm, kết quả đánh giá sẽ không phản ánh đúng năng lực hiểu ngữ nghĩa. Vì vậy, nghiên cứu sử dụng ý tưởng *temporal masking*: che hoặc loại bỏ các tín hiệu thời gian tường minh để kiểm tra xem mô hình có thể suy luận thứ tự sự kiện dựa trên nội dung câu hay không.

Temporal masking giúp tách bài toán thành hai mức năng lực khác nhau. Ở mức đơn giản, mô hình chỉ cần nhận ra rằng năm 1990 xảy ra trước năm 2000. Ở mức khó hơn, mô hình cần hiểu rằng “tham gia kháng chiến” có thể xảy ra trước “được phong tặng danh hiệu”, hoặc “học tại trường đại học” thường xảy ra trước “bảo vệ luận án”. Đây chính là năng lực mà bài toán TEO-TM muốn đánh giá.

1.2.2 Phát biểu bài toán chính thức

Bài toán **Temporal Event Ordering under Temporal Masking** (TEO-TM) được định nghĩa như sau:

Đầu vào: Một cặp câu sự kiện (s_A, s_B) đã được loại bỏ hoặc che đi các biểu thức thời gian tường minh.

Đầu ra: Nhãn phân loại $y \in \{\text{BEFORE}, \text{AFTER}\}$, trong đó $y = \text{BEFORE}$ nếu sự kiện A xảy ra trước sự kiện B , và $y = \text{AFTER}$ nếu sự kiện A xảy ra sau sự kiện B .

Ground truth: Thứ tự đúng được xác định dựa trên năm hoặc mốc thời gian gán nhãn

trong bộ dữ liệu gốc trước khi masking.

Ký hiệu toán học: Cho tập dữ liệu $\mathcal{D} = \{(s_A^{(i)}, s_B^{(i)}, y^{(i)})\}_{i=1}^N$, mục tiêu là học hàm phân loại $f : (s_A, s_B) \mapsto y$ sao cho dự đoán của mô hình gần nhất với nhãn đúng. Với bài toán phân loại cặp sự kiện, một độ đo cơ bản là Accuracy:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[f(s_A^{(i)}, s_B^{(i)}) = y^{(i)}]. \quad (1.1)$$

Tuy nhiên, mục tiêu cuối cùng của nghiên cứu không chỉ là dự đoán đúng từng cặp riêng lẻ, mà còn là khôi phục thứ tự của một chuỗi nhiều sự kiện thuộc cùng một nhân vật. Vì vậy, ở các chương sau, báo cáo sử dụng thêm các độ đo xếp hạng như Kendall Tau, Spearman's Rho và Exact Match để đánh giá mức độ đúng của toàn bộ chuỗi sự kiện.

1.2.3 Các thiết lập đánh giá trong báo cáo

Để đánh giá tác động của mức độ thiếu hụt thông tin thời gian, báo cáo sử dụng ba thiết lập chính:

- **Absolute Time (AT):** giữ nguyên các mốc thời gian trong câu. Thiết lập này cho biết hiệu quả tối đa khi mô hình có đầy đủ tín hiệu thời gian tường minh.
- **Partially Masked Time (PM):** che một phần mốc thời gian, ví dụ chỉ một số sự kiện trong chuỗi bị thay bằng thẻ [MASK]. Thiết lập này mô phỏng văn bản có thông tin thiếu cục bộ.
- **Masked Time (MT):** che toàn bộ mốc thời gian trong các sự kiện. Đây là thiết lập khó nhất vì mô hình phải dựa gần như hoàn toàn vào ngữ nghĩa và tri thức nền.

Ba thiết lập này giúp phân biệt rõ hai loại năng lực: khai thác mốc thời gian tường minh và suy luận thứ tự khi mốc thời gian không còn xuất hiện. Sự chênh lệch kết quả giữa AT, PM và MT cũng phản ánh mức độ phụ thuộc của từng mô hình vào thông tin năm/tháng trực tiếp.

1.2.4 Các thách thức chính

Bài toán TEO-TM có một số thách thức nổi bật:

- **Mơ hồ ngữ nghĩa:** nhiều câu sự kiện ngắn, ít ngữ cảnh, khiến mô hình khó xác định thứ tự nếu không có mốc năm.

- **Sự kiện đồng thời hoặc gần nhau:** hai sự kiện có thể diễn ra trong cùng năm, cùng giai đoạn hoặc không có ranh giới thời gian rõ ràng.
- **Phụ thuộc kiến thức nền:** một số thứ tự chỉ có thể suy ra khi biết tri thức lịch sử, quy luật nghề nghiệp hoặc bối cảnh xã hội.
- **Đặc thù tiếng Việt:** tiếng Việt có cách diễn đạt linh hoạt, ít biến hình từ, và nhiều cụm từ thời gian phụ thuộc vào ngữ cảnh.
- **Nhiều dữ liệu tự động:** dữ liệu thu thập từ Wikipedia có thể chứa câu giới thiệu, câu thiếu chủ ngữ, hoặc mốc thời gian không gắn trực tiếp với sự kiện chính.

Những thách thức này cho thấy TEO-TM không đơn thuần là bài toán trích xuất thông tin, mà là bài toán kết hợp giữa tiền xử lý dữ liệu, biểu diễn ngôn ngữ, học xếp hạng và đánh giá chuỗi sự kiện.

1.3 Các nghiên cứu liên quan

1.3.1 Các nghiên cứu trên thế giới

Bài toán sắp xếp sự kiện thời gian đã thu hút sự quan tâm lớn từ cộng đồng NLP quốc tế trong nhiều năm. Các nghiên cứu ban đầu tập trung vào nhận diện biểu thức thời gian và quan hệ giữa sự kiện với mốc thời gian; về sau, trọng tâm dần mở rộng sang suy luận quan hệ giữa các sự kiện và xây dựng dòng thời gian tự động.

Tài nguyên và chuẩn đánh giá

TimeML [1] là một trong những chuẩn chú thích quan trọng nhất cho thông tin thời gian trong văn bản tiếng Anh. Chuẩn này định nghĩa cách đánh dấu biểu thức thời gian, sự kiện và quan hệ giữa chúng, tạo nền tảng cho nhiều bộ dữ liệu và hệ thống xử lý thời gian sau này.

Dựa trên TimeML, các cuộc thi **TempEval-1/2/3** (2007–2013) [2] thúc đẩy nghiên cứu về nhận dạng biểu thức thời gian, nhận dạng sự kiện và phân loại quan hệ thời gian. Các bài toán trong TempEval giúp chuẩn hóa cách đánh giá hệ thống, nhưng chủ yếu tập trung vào tiếng Anh và một số ngôn ngữ có tài nguyên phong phú.

Bộ dữ liệu **MATRES** [3] tiếp tục cải tiến hướng chú thích quan hệ thời gian bằng cách tập trung vào cấu trúc thời gian của động từ và các trục thời gian chính trong văn bản. Những tài nguyên này cho thấy tầm quan trọng của dữ liệu chú thích chất lượng cao đối với bài toán TEO.

Phương pháp học máy và học sâu

Trước thời kỳ học sâu, các phương pháp dựa trên đặc trưng thủ công như SVM, MaxEnt thường được sử dụng cho phân loại quan hệ thời gian [4]. Các đặc trưng phổ biến gồm từ vựng xung quanh sự kiện, loại động từ, vị trí câu, tín hiệu thời gian và quan hệ cú pháp. Ưu điểm của nhóm phương pháp này là dễ giải thích, nhưng hiệu quả phụ thuộc mạnh vào chất lượng đặc trưng và bộ phân tích ngôn ngữ.

Sự xuất hiện của BERT [5] và các mô hình Transformer đã thay đổi cách tiếp cận bài toán. Thay vì thiết kế đặc trưng thủ công, mô hình có thể học biểu diễn ngữ cảnh của câu và cặp câu trực tiếp từ dữ liệu. Các biến thể như RoBERTa, T5 hoặc các mô hình ngôn ngữ lớn tiếp tục cải thiện khả năng đọc hiểu, suy luận và tổng quát hóa trên nhiều tác vụ NLP.

Suy luận thường thức về thời gian

Một hướng nghiên cứu gần với đề tài là *temporal commonsense reasoning*, tức suy luận thứ tự sự kiện dựa trên tri thức thông thường. Zhou et al. [6] nghiên cứu khả năng suy luận các quan hệ như “nấu ăn” thường xảy ra trước “ăn”, hoặc “thi đỗ” thường xảy ra trước “nhập học”. Hướng này cho thấy nhiều quan hệ thời gian không thể giải quyết chỉ bằng mốc ngày tháng, mà cần đến kiến thức nền và ngữ nghĩa sự kiện.

Bài toán TEO-TM trong báo cáo này có điểm tương đồng với temporal commonsense reasoning, nhưng tập trung vào văn bản tiếng Việt và miền dữ liệu tiểu sử nhân vật. Điều này đặt ra yêu cầu riêng về thu thập dữ liệu, xử lý tiếng Việt và đánh giá mô hình trong bối cảnh thiếu tài nguyên chuẩn.

1.3.2 Các nghiên cứu trong nước

Tài nguyên NLP tiếng Việt

Đối với tiếng Việt, nhiều tài nguyên và công cụ NLP đã được phát triển trong những năm gần đây, tạo điều kiện cho các nghiên cứu học sâu. **PhoBERT** [7] là mô hình BERT đơn ngữ được tiền huấn luyện trên kho văn bản tiếng Việt quy mô lớn, đạt kết quả tốt trên nhiều tác vụ như phân tích cú pháp phụ thuộc, nhận dạng thực thể có tên (NER) và phân loại cảm xúc.

Bên cạnh đó, **underthesea** là bộ công cụ NLP tiếng Việt mã nguồn mở hỗ trợ tách từ, gán nhãn từ loại (POS), nhận dạng thực thể và một số tác vụ tiền xử lý khác. Những công cụ này đặc biệt hữu ích trong giai đoạn xây dựng dữ liệu, ví dụ lọc câu có động từ hoặc chuẩn hóa văn bản trước khi đưa vào mô hình.

Khoảng trống nghiên cứu

Mặc dù tiếng Việt đã có các mô hình ngôn ngữ mạnh và một số công cụ tiền xử lý, các tài nguyên chuyên biệt cho thông tin thời gian vẫn còn hạn chế. Đến thời điểm thực hiện nghiên cứu này, chưa có bộ dữ liệu tiêu chuẩn quy mô lớn cho bài toán TEO trong tiếng Việt, và cũng chưa có benchmark rõ ràng cho trường hợp mốc thời gian bị che đi.

Khoảng trống này dẫn đến hai khó khăn. Thứ nhất, các mô hình không có tập dữ liệu phù hợp để học quan hệ thứ tự sự kiện trong tiếng Việt. Thứ hai, cộng đồng nghiên cứu thiếu một điểm chuẩn chung để so sánh các phương pháp khác nhau. Do đó, việc xây dựng bộ dữ liệu TEO-TM tiếng Việt và đánh giá nhiều nhóm mô hình trên cùng điều kiện thực nghiệm là đóng góp cần thiết của báo cáo.

1.4 Mục tiêu đề tài

Nghiên cứu này tập trung vào hai mục tiêu chính: xây dựng tài nguyên dữ liệu và xây dựng mô hình giải quyết bài toán.

1.4.1 Mục tiêu 1: Xây dựng bộ dữ liệu

Mục tiêu đầu tiên là xây dựng bộ dữ liệu tiếng Việt phục vụ bài toán TEO-TM, bao gồm các sự kiện liên quan đến nhân vật lịch sử và đương đại được thu thập từ Wikipedia tiếng Việt. Bộ dữ liệu cần thỏa mãn một số yêu cầu:

- có quy mô đủ lớn để huấn luyện và đánh giá mô hình học sâu;
- mỗi sự kiện có thể gắn với mốc thời gian hoặc thứ tự đáng tin cậy;
- dữ liệu được lọc để loại bỏ câu nhiễu, câu quá ngắn, câu thiếu thông tin hoặc không mô tả sự kiện;
- quy trình xây dựng có khả năng tái sử dụng và mở rộng cho các nguồn dữ liệu khác.

Để đạt mục tiêu này, báo cáo thiết kế pipeline tự động gồm các bước thu thập URL, xử lý danh sách trang, trích xuất câu sự kiện, lọc chất lượng và gộp dữ liệu. Quy trình chi tiết được trình bày trong Chương 2.

1.4.2 Mục tiêu 2: Xây dựng và đánh giá mô hình

Mục tiêu thứ hai là xây dựng mô hình có khả năng suy luận thứ tự giữa các sự kiện tiếng Việt khi thông tin thời gian bị che. Báo cáo tập trung vào hướng tiếp cận so sánh từng cặp sự kiện bằng PhoBERT, sau đó tổng hợp các so sánh cặp thành thứ tự toàn cục bằng thuật toán xếp hạng.

Việc đánh giá mô hình được thực hiện trên nhiều thiết lập dữ liệu khác nhau nhằm trả lời ba câu hỏi nghiên cứu:

1. Mô hình đạt hiệu quả như thế nào khi có đầy đủ mốc thời gian tường minh?
2. Hiệu quả suy giảm ra sao khi một phần hoặc toàn bộ mốc thời gian bị che?
3. Mô hình fine-tune chuyên biệt có ưu thế gì so với prompting trực tiếp bằng các mô hình ngôn ngữ lớn?

Các câu hỏi này giúp báo cáo không chỉ đo hiệu suất cuối cùng, mà còn phân tích được bản chất năng lực suy luận thời gian của mô hình.

1.5 Phạm vi nghiên cứu

Báo cáo giới hạn phạm vi ở văn bản tiếng Việt thuộc miền tiểu sử nhân vật trên Wikipedia. Đây là miền dữ liệu phù hợp vì các bài viết tiểu sử thường chứa nhiều sự kiện có thứ tự tự nhiên như sinh ra, học tập, làm việc, giữ chức vụ, nhận giải thưởng và qua đời. Tuy nhiên, phạm vi này cũng có giới hạn: kết quả chưa nhất thiết tổng quát hoàn toàn sang tin tức, văn bản pháp lý, truyện kể hoặc tài liệu lịch sử dài.

Báo cáo tập trung vào quan hệ BEFORE/AFTER giữa các sự kiện, chưa mở rộng sang các quan hệ phức tạp hơn như SIMULTANEOUS, INCLUDES hoặc OVERLAP. Ngoài ra, thứ tự đúng được xác định chủ yếu dựa trên mốc thời gian trích xuất được từ dữ liệu gốc; vì vậy các trường hợp mốc thời gian mơ hồ hoặc nhiều sự kiện xảy ra cùng năm vẫn là nguồn khó khăn cần được xử lý sâu hơn trong các nghiên cứu tiếp theo.

1.6 Tổng kết chương

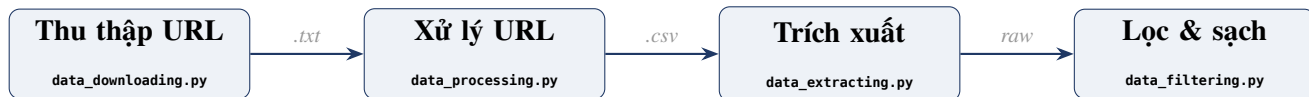
Chương 1 đã trình bày cơ sở lý thuyết và phát biểu chính thức của bài toán TEO-TM. Nội dung chương đã làm rõ khái niệm thông tin thời gian, phân loại các dạng biểu hiện thời gian trong tiếng Việt, động cơ của temporal masking, các thiết lập đánh giá, những thách thức chính và

khoảng trống nghiên cứu trong bối cảnh tiếng Việt. Trên cơ sở đó, chương xác định hai mục tiêu trọng tâm của đề tài: xây dựng bộ dữ liệu TEO-TM tiếng Việt quy mô lớn và phát triển mô hình có khả năng sắp xếp chuỗi sự kiện khi thông tin thời gian bị che. Chương 2 tiếp theo sẽ trình bày chi tiết quy trình xây dựng bộ dữ liệu phục vụ cho các thực nghiệm của báo cáo.

CHƯƠNG 2

XÂY DỰNG BỘ DỮ LIỆU SẮP XẾP SỰ KIẾN THỜI GIAN TRONG TIẾNG VIỆT

Bộ dữ liệu được xây dựng theo pipeline tự động gồm năm giai đoạn, minh họa trong Hình 2.1. Toàn bộ mã nguồn được viết bằng Python 3.10+ và được trình bày cụ thể trong các mục dưới đây.



Hình 2.1. Pipeline xây dựng bộ dữ liệu TEO-TM tiếng Việt

2.1 Xây dựng dữ liệu thủ công

2.1.1 Giai đoạn 1: Thu thập URL Wikipedia ([data_downloading.py](#))

Thiết kế thuật toán

Hệ thống thu thập URL sử dụng chiến lược **BFS có giới hạn độ sâu** (bounded BFS) để duyệt các danh mục Wikipedia tiếng Việt. Hai nhóm nhân vật được thu thập song song:

- **Nhân vật nổi tiếng:** 10 danh mục gốc (chính khách, nhà khoa học, nghệ sĩ...), mục tiêu 20.000 URL, độ sâu tối đa 3.
- **Nhân vật ít biết đến:** 8 danh mục chuyên biệt (nhà côn trùng học, nhà ngôn ngữ học...), mục tiêu 2.000 URL, độ sâu tối đa 2.

Thuật toán được trình bày trong Thuật toán 1.

Algorithm 1: BFS thu thập URL nhân vật Wikipedia

Input: Danh sách danh mục gốc $\mathcal{C}_0$, số URL mục tiêu T , độ sâu tối đa D

Output: Tập URL nhân vật $\mathcal{U}$

```

1  $\mathcal{U} \leftarrow \emptyset$ ;
2 queue  $\leftarrow [(c, 0) \text{ for } c \in \mathcal{C}_0]$ ;
3 visited  $\leftarrow \emptyset$ ;
4 while queue  $\neq \emptyset$  and  $|\mathcal{U}| < T$  do
5     (cat, depth)  $\leftarrow$  queue.popleft();
6     if cat  $\in$  visited or depth  $> D$  then
7         continue;
8     visited.add(cat);
9     foreach item  $\in$  fetchCategoryMembers(cat) do
10         if item.ns = 0 and isValidPerson(item.title) then
11              $\mathcal{U} \leftarrow \mathcal{U} \cup \{\text{buildURL}(\text{item.title})\}$ ;
12         else if item.ns = 14 then
13             queue.append((item.title, depth + 1));
14 return  $\mathcal{U}$ 

```

Mã nguồn chi tiết

```

1 API_URL = "https://vi.wikipedia.org/w/api.php"
2 HEADERS = {"User-Agent": "WikiDataFetcherBot/4.0 (research)"}
3 REQUEST_DELAY = 0.15 # Giây nghỉ ngơi giữa các request
4 MAX_RETRIES = 3 # Số lần thử lại khi thất bại
5
6 def is_valid_person_page(title: str) -> bool:
7     """Loại bỏ trang không phải bài viết nhân vật."""
8     t = title.lower()
9     bad = ["danh sach", "dinh huong", "wikipedia:",
10           "cong thong tin:", "tap tin:", "template:"]
11     return not any(x in t for x in bad)
12
13 def fetch_category_members(category: str,
14                             cmcontinue=None) -> dict:
15     """Gửi MediaWiki API lấy danh sách thành viên danh mục."""
16

```

```

17     S  ụngng exponential backoff khi  ấtht  ặbi:
18     attempt 0 -> sleep ~1s
19     attempt 1 -> sleep ~2s
20     attempt 2 -> sleep ~4s (+jitter  ẫng ngẫu nhiên)
21     """
22     params = {
23         "action": "query", "list": "categorymembers",
24         "cmtitle": category, "cmlimit": "max",
25         "cmnamespace": "0|14",    # 0=bai viet, 14=danh muc
26         "format": "json",
27     }
28     if cmcontinue:
29         params["cmcontinue"] = cmcontinue
30
31     for attempt in range(MAX_RETRIES):
32         try:
33             r = requests.get(API_URL, params=params,
34                             headers=HEADERS, timeout=20)
35             if r.status_code == 200:
36                 return r.json()
37         except Exception:
38             pass
39         time.sleep((2 ** attempt) + random.random())
40     return None

```

Listing 2.1. Bộ lọc trang hợp lệ và hàm gọi API Wikipedia ([data_downloading.py](#))

```

1  def get_wiki_urls_deep_crawl(seed_categories: list,
2                                target_count: int,
3                                max_depth: int = 3) -> list:
4      """BFS có ớgii  ặnh độ sâu, thu  ặthp URL nh  ặvt.ê'
5
6      Chin ượlc random.shuffle ượtrc khi ệduyt ừtng batchđả
7      m  ặbo tính đa ặngng khi ừđng ớsm ở target_count.
8      """
9      urls = set()
10     processed = set()
11     queue = deque([(cat, 0) for cat in seed_categories])
12
13     while queue and len(urls) < target_count:

```

```

14     current_cat, depth = queue.popleft()
15     if current_cat in processed or depth > max_depth:
16         continue
17     processed.add(current_cat)
18
19     cmcontinue = None
20     while len(urls) < target_count:
21         data = fetch_category_members(current_cat, cmcontinue)
22         if not data:
23             break
24
25         members = data.get("query", {}).get(
26             "categorymembers", [])
27         random.shuffle(members) # Đảm bảo đa dạng
28
29         for item in members:
30             if item["ns"] == 0: # Bài viết
31                 if is_valid_person_page(item["title"]):
32                     urls.add(build_wiki_url(item["title"]))
33             elif item["ns"] == 14: # Danh mục con
34                 queue.append((item["title"], depth + 1))
35             if len(urls) >= target_count:
36                 break
37
38         cmcontinue = (data.get("continue", {})
39                       .get("cmcontinue"))
40         if not cmcontinue:
41             break
42         time.sleep(REQUEST_DELAY)
43
44     return list(urls)[:target_count]
45
46 # ===== ACHY CRAWL =====
47 urls_famous = get_wiki_urls_deep_crawl(cats_famous, 20000, 3)
48 urls_obscura = get_wiki_urls_deep_crawl(cats_obscura, 2000, 2)
49 all_urls = list(set(urls_famous + urls_obscura))

```

Listing 2.2. Hàm BFS chính thu thập URL ([data_downloading.py](#))

Bảng 2.1. Thống kê quá trình thu thập URL

Thông số	Giá trị
Tổng URL thu thập (nhân vật nổi tiếng)	≈ 20.000
Tổng URL thu thập (nhân vật ít biết)	≈ 2.000
Tổng URL sau gộp và loại trùng	≈ 22.000
Tốc độ crawl trung bình	$\approx 6-7$ URL/giây
Thời gian ước tính	$\approx 55-60$ phút

2.1.2 Giai đoạn 2: Xử lý URL (`data_processing.py`)

File này chuyển đổi `wiki_people_urls.txt` sang định dạng CSV có cấu trúc, thêm cột STT, và loại bỏ các ký tự lạ <> có thể xuất hiện do lỗi encoding.

```

1 with open("wiki_people_urls.txt", "r", encoding="utf-8") as f:
2     all_urls = f.readlines()
3
4 cleaned_urls = []
5 for url in all_urls:
6     url = url.strip()
7     if not url:
8         continue                # Bỏ dòng rỗng
9     url = url.replace("<", "").replace(">", "") # Xóa ký tự lạ
10    cleaned_urls.append(url)
11
12 df = pd.DataFrame(cleaned_urls, columns=["Link Wikipedia"])
13 df.insert(0, "STT", range(1, len(df) + 1))
14 df.to_csv("wiki_people_urls.csv", index=False,
15          encoding="utf-8-sig")    # BOM: Excel đọc được

```

Listing 2.3. Làm sạch URL và xuất ra CSV (`data_processing.py`)

2.1.3 Giai đoạn 3: Trích xuất sự kiện (`data_extracting.py`)

Quy trình xử lý

Từ danh sách URL, hệ thống gọi Wikipedia API để lấy nội dung bài viết dạng plain text, thực hiện ba bước: (1) làm sạch văn bản, (2) tách câu, (3) lọc câu chứa thông tin thời gian.

```

1 # Regex khop nam trong khoang 1000-2029
2 YEAR_PATTERN = re.compile(r'\b(1[0-9]{3}|20[0-2][0-9])\b')
3
4 session = requests.Session()
5 session.headers.update({"User-Agent": "SentenceExtractorBot/3.0"})
6
7 for index, title in enumerate(titles, start=1):
8     params = {"action": "query", "prop": "extracts",
9               "titles": title, "explaintext": 1, "format": "json"}
10    res = session.get(WIKI_API_URL, params=params, timeout=15)
11
12    for page_id, page_info in (res.json()
13                               .get("query", {}).
14                               .get("pages", {}).items()):
15        extract = page_info.get("extract", "")
16        if not extract: continue
17
18        # Buoc 1: Loai tieu de section (==Ten section==)
19        clean = re.sub(r'==+.*?==+', '', extract)
20
21        # Buoc 2: Tach cau theo dau ket thuc .!?
22        sentences = re.split(r'(?<=[.!?])\s+', clean)
23
24        person_events = []
25        seen = set()
26        for s in sentences:
27            s = s.strip().replace('\n', ' ')
28            if len(s) < 20 or s in seen: continue
29
30            # Buoc 3: Chi giu cau chua nam
31            m = YEAR_PATTERN.search(s)
32            if m:
33                person_events.append((int(m.group(1)), s))
34                seen.add(s)
35
36        # Sap xep theo nam tang dan
37        person_events.sort(key=lambda x: x[0])
38        for year, sent in person_events:
39            all_results.append({

```

```

40         "Nhan vat": title.replace('_', ' '),
41         "Nam": year,
42         "Su kien chi tiet": sent
43     })
44
45     time.sleep(1)    # Rate limiting bat buoc

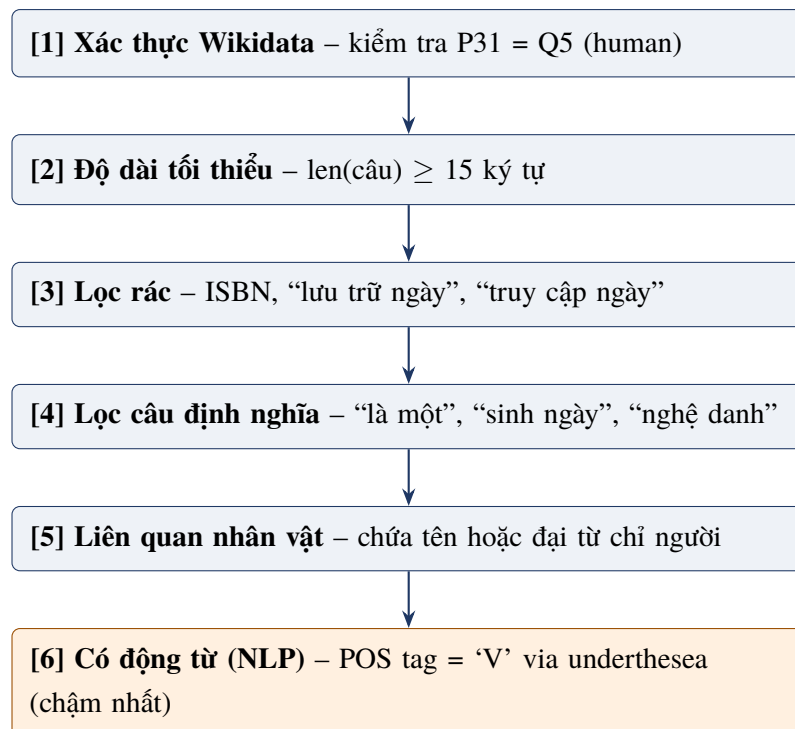
```

Listing 2.4. Trích xuất sự kiện có mốc năm từ Wikipedia ([data_extracting.py](#))

Lưu ý về regex năm: Pattern `\b(1[0-9]{3}|20[0-2][0-9])\b` khớp năm từ 1000 đến 2029. Ranh giới từ `\b` ngăn khớp nhầm với số điện thoại, mã ISBN, hay số trang.

2.1.4 Giai đoạn 4: Lọc và làm sạch ([data_filtering.py](#))

Đây là giai đoạn phức tạp nhất, áp dụng sáu lớp lọc theo thứ tự từ nhanh đến chậm (Hình 2.2).



Hình 2.2. Pipeline lọc dữ liệu sáu lớp trong [data_filtering.py](#)

Lớp 1: Xác thực nhân vật qua Wikidata

```

1 def is_valid_person_name(name: str) -> bool:
2     """êKim tra nhân âvt có âphi ườngi âtht (Q5 - human).
3

```

```

4      Truy vấn Wikidata theo thứ tự ưu tiên: vi -> zh -> en.
5      Cache kết quả vào wikidata_cache.json để tránh gọi lại.
6      Thực hiện kiểm tra:
7          P31 (instance of) = Q5 (human)
8      """
9      if name in wikidata_cache:
10         return wikidata_cache[name]
11
12     for lang in ["vi", "zh", "en"]:
13         params = {"action": "wbsearchentities", "format": "json",
14                  "language": lang, "search": name, "limit": 5}
15         try:
16             data = requests.get(
17                 "https://www.wikidata.org/w/api.php",
18                 params=params, timeout=5).json()
19
20             for item in data.get("search", []):
21                 eid = item["id"]
22                 detail = requests.get(
23                     f"https://www.wikidata.org/wiki/"
24                     f"Special:EntityData/{eid}.json",
25                     timeout=5).json()
26                 claims = (detail["entities"][eid]
27                           .get("claims", {}))
28                 for claim in claims.get("P31", []):
29                     val = (claim.get("mainsnak", {})
30                           .get("datavalue", {})
31                           .get("value", {})).get("id"))
32                     if val == "Q5":          # Q5 = human
33                         wikidata_cache[name] = True
34                         return True
35             time.sleep(0.2)
36         except:
37             continue
38
39     wikidata_cache[name] = False
40     return False

```

Listing 2.5. Xác thực nhân vật là người thật (Q5) qua Wikidata API

Lớp 2–5: Bộ lọc nhanh (Regex/String)

```

1 def is_garbage_line(text: str) -> bool:
2     """Loại bỏ dòng chứa thông tin uth ụmc/tài liệu tham khảo."""
3     kws = ['isbn', 'issn', 'wayback machine', 'lưu trữ ngay',
4           'nhà xuất bản', 'truy cập ngay']
5     return any(w in text.lower() for w in kws)
6
7 def is_intro_definition_sentence(text: str,
8                                 person_name: str) -> bool:
9     """Nhận định câu định nghĩa nhân vật (không phải ựs ệkin)."""
10    patterns = [
11        r'sinh ngày .* là',      # "sinh ngày ... là"
12        r'là một\s+',           # "là một [danh từ]"
13        r'là (nam|nu)\s+',       # "là nam/ữn ..."
14        r'là nhà\s+',           # "là nhà ..."
15        r'nghe danh',           # "nghệ danh"
16        r'tên khai sinh',       # "tên khai sinh"
17    ]
18    tl = text.lower()
19    return any(re.search(p, tl) for p in patterns)
20
21 def is_related(text: str, person_name: str) -> bool:
22     """Câu ảphi đề ậ cp đến nhân vật (tên ặhoc đại từ)."""
23     tl = text.lower()
24     main = re.sub(r'\(.*\)', '', person_name).strip().lower()
25     # Lay các từ có nghĩa (do dài > 2)
26     parts = [p for p in main.split() if len(p) > 2]
27     pronouns = ['ông', 'bà', 'anh', 'chị', 'hà', 'y', 'vi']
28     return any(f" {kw} " in f" {tl} "
29               for kw in [main] + parts + pronouns)

```

Listing 2.6. Các bộ lọc nhanh dựa trên regex và string matching**Lớp 6: Kiểm tra động từ bằng POS Tagging**

```

1 from underthesea import sent_tokenize, pos_tag
2
3 def has_verb(text: str) -> bool:
4     """Câu ảphi chứa ít ấnh ộmt động từ (tag = 'V').Đ

```



```

5
6     đây là bộ lọc để đảm bảo rằng nên được đặt sau cùng trong pipeline.
7     Loại bỏ các câu chỉ là danh sách, tiêu đề, hoặc mô tả ngắn.
8     """
9     try:
10         return any(tag == 'V' for _, tag in pos_tag(text))
11     except:
12         return False
13
14 # ===== ỨNG DỤNG TOÀN BỘ PIPELINE =====
15 for idx, row in df.iterrows():
16     name = str(row['Nhân vật'])
17     if name not in valid_people: continue # Lọc Wikidata
18
19     for s in sent_tokenize(str(row['Sự kiện'])):
20         s = clean_sentence_start(s)
21         # --- Bỏ lọc nhanh (O(n) string) ---
22         if len(s) < 15: continue
23         if is_garbage_line(s): continue
24         if is_intro_definition_sentence(s, name): continue
25         if not is_related(s, name): continue
26         # --- Bỏ lọc chậm (O(n) NLP) ---
27         if has_verb(s):
28             new_rows.append({'Nhân vật': name,
29                             'Tên': row['Tên'], 'Sự kiện': s})
30
31 # Giữ 5-15 sự kiện mỗi nhân vật
32 final_df = (pd.DataFrame(new_rows)
33             .groupby('Nhân vật')
34             .filter(lambda x: len(x) >= 5)
35             .groupby('Nhân vật').head(15))

```

Listing 2.7. Bộ lọc NLP dùng POS tagging của underthesea

2.1.5 Giai đoạn 5: Gộp dữ liệu ([data_merging.py](#))

Do dữ liệu được xử lý song song thành ba batch, giai đoạn gộp thực hiện loại bỏ trùng lặp nhân vật theo nguyên tắc ưu tiên file 1, rồi file 2, rồi file 3.

```

1 df1 = pd.read_csv("data_cleaned_1.csv", encoding="utf-8-sig")

```

```

2 df2 = pd.read_csv("data_cleaned_2.csv", encoding="utf-8-sig")
3 df3 = pd.read_csv("data_cleaned_3.csv", encoding="utf-8-sig")
4
5 for df in [df1, df2, df3]:
6     df["Nhan vat"] = df["Nhan vat"].astype(str).str.strip()
7
8 # File 2: giu lai nhung nhan vat CHUA co trong file 1
9 people_1 = set(df1["Nhan vat"])
10 df2_filtered = df2[~df2["Nhan vat"].isin(people_1)]
11
12 # File 3: giu lai nhung nhan vat CHUA co trong file 1 hoac 2
13 people_12 = people_1.union(set(df2_filtered["Nhan vat"]))
14 df3_filtered = df3[~df3["Nhan vat"].isin(people_12)]
15
16 final_df = pd.concat([df1, df2_filtered, df3_filtered],
17                      ignore_index=True)
18 final_df.to_csv("merged_unique_people.csv", index=False,
19                encoding="utf-8-sig")

```

Listing 2.8. Gộp ba batch, mỗi nhân vật chỉ giữ ở một file (`data_merging.py`)

2.2 Thống kê bộ dữ liệu

Bảng 2.2. Thống kê tổng quan bộ dữ liệu TEO-TM tiếng Việt

Thống kê	Giá trị
Tổng số sự kiện (câu văn)	> 300.000
Số nhân vật duy nhất	> 20.000
Số sự kiện trung bình/nhân vật	5 – 15
Khoảng thời gian bao phủ	Thế kỷ 10 – 2024
Ngôn ngữ	Tiếng Việt
Nguồn dữ liệu	Wikipedia tiếng Việt
Định dạng lưu trữ	CSV (UTF-8 BOM)
Thời gian xây dựng toàn bộ	≈ 3–4 giờ

Bảng 2.3. Phân bố loại nhân vật trong bộ dữ liệu

Loại nhân vật	Danh mục gốc	Nhóm
Chính khách, nguyên thủ quốc gia	2 danh mục	Nổi tiếng
Nhà khoa học, học giả	1 danh mục	Nổi tiếng
Nghệ sĩ (ca sĩ, diễn viên, đạo diễn)	3 danh mục	Nổi tiếng
Vận động viên	1 danh mục	Nổi tiếng
Doanh nhân, nhà văn	2 danh mục	Nổi tiếng
Nhân vật lịch sử	1 danh mục	Nổi tiếng
Chuyên gia lĩnh vực hẹp	8 danh mục	Ít biết đến

2.3 Ngoại lệ bộ dữ liệu

1. **Năm gán nhãn không phải năm sự kiện:** Một số câu chứa năm tham chiếu chứ không phải năm xảy ra sự kiện, ví dụ “Theo số liệu năm 2020, ông đã...”. Regex không phân biệt được hai trường hợp này.
2. **Câu mô tả nhiều năm:** Pattern chỉ lấy năm đầu tiên. Câu “Từ năm 1995 đến 2005...” sẽ được gán nhãn năm 1995.
3. **Sai lệch từ Wikipedia:** Nội dung có thể chứa lỗi dịch thuật, thông tin chưa kiểm chứng, hoặc vandalism chưa được xử lý.
4. **Phân bố không đồng đều:** Nhân vật phương Tây chiếm tỷ lệ cao hơn do mức độ bao phủ của Wikipedia tiếng Việt không đồng đều.
5. **Câu giới thiệu lọt qua:** Bộ lọc dựa trên regex cố định không bắt được tất cả các biến thể câu giới thiệu.
6. **Rate-limit Wikidata:** Có thể dẫn đến bỏ qua xác thực một số nhân vật, gây nhiễu dữ liệu nhỏ.

2.4 Tổng kết chương

Chương 2 đã trình bày chi tiết quy trình năm giai đoạn xây dựng bộ dữ liệu cùng toàn bộ mã nguồn Python. Các thiết kế kỹ thuật quan trọng bao gồm BFS với random shuffle, bộ lọc

sáu lớp từ nhanh đến chậm, xác thực Wikidata với cache cục bộ, và xử lý song song ba batch. Chương 3 tiếp theo sẽ trình bày các mô hình thực nghiệm trên bộ dữ liệu này.

CHƯƠNG 3

XÂY DỰNG MÔ HÌNH VÀ THUẬT TOÁN SUY LUẬN

Khác với các phương pháp trích xuất mốc thời gian trực tiếp (Information Extraction) hoặc phân loại thứ tự bằng hồi quy (Regression), nghiên cứu này tiếp cận bài toán sắp xếp chuỗi sự kiện thời gian (Temporal Event Ordering - TEO) dưới góc độ **Xếp hạng cặp (Pairwise Ranking)**. Thay vì yêu cầu hệ thống đọc toàn bộ danh sách sự kiện và dự đoán một mảng thứ tự toàn cục, bài toán được phân rã thành nhiều phép thử so sánh nhị phân cục bộ.

Toàn bộ quy trình định dạng dữ liệu, kiến trúc mô hình và thuật toán khôi phục chuỗi thời gian được triển khai trên nền tảng PyTorch và thư viện Transformers.

3.1 Tiền xử lý và định dạng dữ liệu đầu vào

3.1.1 Khung suy luận ngôn ngữ tự nhiên (NLI Framing)

Để tối ưu hóa khả năng hiểu ngữ nghĩa của mô hình ngôn ngữ lớn, các cặp sự kiện không được đưa vào mô hình dưới dạng văn bản thô ráp. Thay vào đó, chúng được đóng gói theo định dạng của bài toán Suy luận Ngôn ngữ Tự nhiên (Natural Language Inference - NLI), bao gồm hai phần: **Tiền đề (Premise)** và **Giả thuyết (Hypothesis)**.

Cụ thể, Tiền đề sẽ cung cấp bối cảnh (tên nhân vật) và nội dung của hai sự kiện X và Y . Giả thuyết luôn là một câu khẳng định tĩnh: "*Sự kiện X xảy ra trước Sự kiện Y .*". Bằng cách này, mô hình bị buộc phải đối chiếu nội dung của Tiền đề để đưa ra điểm số đánh giá mức độ hợp lý của Giả thuyết. Cơ chế này đảm bảo tính trung lập và ngăn chặn rò rỉ nhãn (label leakage) trong quá trình huấn luyện.

```
1 def build_prompt(char_name, event_a, event_b):
2     """
3     Tr ên (premise, hypothesis) theo format NLI trung ập.
4     event_a là ự s ệkin đượ ặgi định ặxy ra ướtrc event_b.
5     Hypothesis ữgi nguyên – model ộbuc ặphi ựda vào ộni dung premise.
6     """
```

```

7     premise = (
8         f"êTiu ưS {char_name}. "
9         f"ưS ệkin X: {event_a} | ưS ệkin Y: {event_b}"
10    )
11    hypothesis = "ưS ệkin X ảxy ra ưótrc ưS ệkin Y."
12    return premise, hypothesis

```

Listing 3.1. Hàm đóng gói dữ liệu theo định dạng NLI

3.1.2 Lấy mẫu theo cửa sổ trượt (Window Sampling)

Với một danh sách gồm N sự kiện của một nhân vật, số lượng cặp so sánh tối đa có thể tạo ra là tổ hợp $O(N^2)$. Tuy nhiên, việc so sánh hai sự kiện có khoảng cách thời gian quá lớn trong cuộc đời nhân vật thường không mang lại nhiều tín hiệu tối ưu hóa hữu ích, do ngữ nghĩa của chúng đã quá khác biệt.

Do đó, thuật toán áp dụng cơ chế Cửa sổ trượt (Window Sampling) với kích thước $W = 4$. Quá trình tạo dữ liệu huấn luyện chỉ ghép cặp một sự kiện với tối đa 4 sự kiện xảy ra ngay sau nó. Mỗi cặp sự kiện sẽ sinh ra hai mẫu dữ liệu:

- **Mẫu thuận (Positive):** Sự kiện X thực sự xảy ra trước sự kiện Y , nhãn mục tiêu là 1.
- **Mẫu nghịch (Negative):** Đảo ngược vị trí sự kiện Y và X , nhãn mục tiêu là 0.

3.2 Kiến trúc mô hình và Chiến lược học tập

3.2.1 Tinh chỉnh PhoBERT với lớp phân loại vô hướng

Mô hình lõi được sử dụng là **PhoBERT** (vinai/phobert-base-v2). Khác với các bài toán phân loại thông thường sử dụng hàm Softmax để xuất ra xác suất đa lớp, đầu ra của PhoBERT trong nghiên cứu này được cấu hình với num_labels=1. Lớp hồi quy tuyến tính cuối cùng của mô hình sẽ xuất ra một giá trị vô hướng duy nhất (scalar logit), biểu diễn đánh giá của mô hình về việc sự kiện X xảy ra trước sự kiện Y hay ngược lại.

Nhằm tăng tốc độ hội tụ và ngăn ngừa hiện tượng học vẹt (overfitting) trên tập dữ liệu hẹp, kỹ thuật **Đóng băng trọng số (Layer Freezing)** được áp dụng. Sáu lớp biến đổi (encoder layers) đầu tiên của kiến trúc RoBERTa được đóng băng hoàn toàn (requires_grad = False). Mô hình chỉ cập nhật gradient ở các lớp trên cùng, nơi chịu trách nhiệm tổng hợp ngữ nghĩa bậc cao phục vụ tác vụ xếp hạng.

```

1 model = AutoModelForSequenceClassification.from_pretrained(
2     "vinai/phobert-base-v2", num_labels=1).to(device)
3
4 # Đóng băng 6 lớp encoder đầu tiên
5 for module in model.roberta.encoder.layer[:6]:
6     for param in module.parameters():
7         param.requires_grad = False

```

Listing 3.2. Khởi tạo mô hình và đóng băng trọng số

3.2.2 Tối ưu hóa với Margin Ranking Loss

Hệ thống sử dụng hàm mất mát `MarginRankingLoss` được thiết kế chuyên biệt cho các bài toán Learning-to-Rank. Cơ chế hoạt động của hàm này yêu cầu đầu vào là một cặp điểm số: điểm của mẫu thuận (`pos_scores`) và điểm của mẫu nghịch (`neg_scores`).

Với tham số $margin = 1.0$, quá trình lan truyền ngược (backpropagation) sẽ không ép mô hình đưa ra một con số cụ thể, mà chỉ yêu cầu điểm số của mẫu thuận phải lớn hơn điểm số của mẫu nghịch một khoảng cách (biên độ) ít nhất bằng 1.0. Nếu mô hình đạt được biên độ này, loss sẽ bằng 0; ngược lại, mô hình sẽ bị phạt tỷ lệ thuận với mức độ vi phạm biên độ.

3.3 Thuật toán Borda Count khôi phục chuỗi thời gian

Trong giai đoạn kiểm thử (Inference), mô hình chỉ có khả năng so sánh từng cặp sự kiện, do đó cần một thuật toán hậu xử lý để khôi phục lại chuỗi thời gian toàn cục gồm N sự kiện. Hệ thống sử dụng một biến thể của thuật toán bầu cử **Borda Count** với cơ chế triệt tiêu thiên kiến vị trí (position bias). Quy trình được mô tả qua các bước sau:

1. **Xáo trộn ngẫu nhiên (Shuffling):** Đầu vào của một nhân vật gồm N sự kiện sẽ được xáo trộn ngẫu nhiên hoàn toàn. Bước này đảm bảo mô hình không dự đoán dựa trên thứ tự xuất hiện gốc trong văn bản.
2. **Lập ma trận đối đầu:** Sinh ra toàn bộ $N \times (N - 1)$ cặp so sánh có thể có từ tập sự kiện đã xáo trộn.
3. **Chấm điểm (Scoring):** Đưa toàn bộ các cặp qua mô hình PhoBERT để thu về điểm logit dự đoán. Khởi tạo một mảng `borda_scores` chứa điểm số của N sự kiện.
4. **Tích lũy điểm:** Với mỗi phép so sánh giữa sự kiện i và j , nếu sự kiện i đóng vai trò là X (xảy ra trước), điểm logit thu được sẽ được cộng tích lũy vào `borda_scores[i]`.

5. **Xếp hạng (Ranking):** Sau khi duyệt qua mọi cặp, các sự kiện được sắp xếp giảm dần dựa trên tổng điểm Borda. Sự kiện "đánh bại" nhiều sự kiện khác nhất với điểm số logit cao nhất sẽ được đặt ở vị trí đầu tiên của chuỗi thời gian.

```

1 borda_scores = np.zeros(n)
2 for k in range(0, len(pairs), batch_size):
3     # Đưa các cặp về batch qua model
4     inputs = tokenizer(batch_pairs_text, return_tensors='pt', ...)
5     with torch.no_grad():
6         scores = model(**inputs).logits.squeeze(-1).cpu().numpy()
7
8     # Cộng tích lũy điểm logit cho sự kiện đứng trước
9     for idx, score in enumerate(scores):
10         i, j = pair_indices[k + idx]
11         borda_scores[i] += score
12
13 # Sắp xếp các sự kiện theo tổng điểm giảm dần
14 pred_order_indices = np.argsort(borda_scores)[::-1]

```

Listing 3.3. Luồng thuật toán chấm điểm Borda Count

3.4 Tổng kết chương

Chương 3 đã trình bày toàn bộ kiến trúc hệ thống và thuật toán cốt lõi nhằm giải quyết bài toán sắp xếp chuỗi sự kiện. Bằng cách kết hợp định dạng dữ liệu NLI trung lập, kỹ thuật đóng băng trọng số một phần trên PhoBERT, sử dụng hàm Margin Ranking Loss và thuật toán bầu cử Borda Count, mô hình được thiết kế để học cách thấu hiểu ngữ cảnh và quan hệ nhân - quả giữa các câu văn thay vì học vẹt. Kiến trúc này tạo tiền đề vững chắc cho các bước đánh giá thực nghiệm và phân tích hiệu năng chi tiết trên các mức độ khó khác nhau của bộ dữ liệu, sẽ được trình bày ở chương tiếp theo.

CHƯƠNG 4

ĐÁNH GIÁ MÔ HÌNH VÀ SO SÁNH THỰC NGHIỆM

4.1 Phương pháp đánh giá và Các hệ đo lường

Nhằm kiểm chứng toàn diện năng lực nhận thức thời gian, khả năng đọc hiểu ngữ nghĩa và suy luận logic chuỗi tự sự của các mô hình, tập dữ liệu kiểm thử được cấu trúc thành ba thiết lập đầu vào với mức độ che lấp thông tin tăng dần: **Chính xác (AT)** - giữ nguyên mốc thời gian; **Kiểm khuyết (MT-1)** - che ngẫu nhiên 1 sự kiện; và **Trống (MT-full)** - che toàn bộ mốc năm.

Để đánh giá chi tiết trên nhiều khía cạnh, nghiên cứu sử dụng hệ thống 6 chỉ số đo lường toàn diện:

1. **Kendall Tau (τ)**: Đánh giá tỷ lệ các cặp sự kiện được xếp đúng thứ tự (đo lường nghịch thế).
2. **Spearman's Rho (ρ)**: Đánh giá độ tương quan tuyến tính của toàn bộ phân phối thứ hạng dự đoán so với thực tế. Điểm này thường cao hơn Tau, cho thấy mô hình có thể nắm được "chiều hướng chung" dù sai lệch một vài cặp cục bộ.
3. **Exact Match (EM %)**: Tỷ lệ các nhân vật có chuỗi sự kiện được khôi phục chính xác 100% không sai một vị trí nào. Đây là chỉ số khắt khe nhất.
4. **Short Acc (%)**: Độ chính xác dự đoán cặp trên nhóm nhân vật có chuỗi sự kiện ngắn (từ 5 đến 10 sự kiện).
5. **Long Acc (%)**: Độ chính xác dự đoán cặp trên nhóm nhân vật có chuỗi sự kiện dài (từ 11 đến 15 sự kiện). Bài toán trở nên phức tạp theo cấp số nhân ($O(n^2)$) khi chuỗi dài ra.
6. **Tỷ lệ ổn định (Ratio L/S)**: Bằng tỷ số giữa *Long Acc* / *Short Acc*. Hệ số này lý tưởng khi tiệm cận 1.0, phản ánh khả năng duy trì sự mạch lạc của mô hình khi độ dài chuỗi tăng lên, không bị suy thoái hiệu suất (context degradation).

4.2 Kết quả đánh giá trên tập xác thực (Validation Metrics)

Trong quá trình huấn luyện mô hình PhoBERT Pairwise, hệ thống liên tục ghi nhận điểm số trên tập xác thực toàn phần tại mỗi vòng lặp (epoch). Bảng 4.1 trình bày kết quả tốt nhất được ghi nhận (hội tụ tại Epoch 3) của ba mô hình khi được đánh giá cùng miền (In-domain).

Bảng 4.1. Kết quả tốt nhất ghi nhận trong quá trình huấn luyện (In-domain Validation)

Mô hình	Kendall Tau	Spearman	EM (%)	Short Acc	Long Acc	Ratio L/S
Model AT	0.8756	0.9331	27.28%	95.04%	93.42%	0.98
Model MT-1	0.7848	0.8757	6.58%	88.51%	90.04%	1.02
Model MT-full	0.4795	0.5852	2.90%	75.14%	72.63%	0.96

Đáng chú ý, **Model MT-1** ghi nhận hệ số ổn định (Ratio L/S) đạt 1.03, cho thấy mô hình xử lý các chuỗi sự kiện dài thậm chí còn tốt hơn các chuỗi ngắn. Đây là lợi thế tuyệt đối của kiến trúc Pairwise: khi chia nhỏ thành các cặp, mô hình không bị ”quá tải” nhận thức độ dài.

4.3 Đánh giá chéo trên tập kiểm thử thu nhỏ (Cross-Evaluation 3x3)

Để đánh giá tính tổng quát hóa độc lập, một tập kiểm thử gồm **250 nhân vật** được thiết lập. Ba phiên bản mô hình lần lượt được đưa vào suy luận chéo trên ba biến thể của tập dữ liệu này, tạo thành ma trận 9 kịch bản thực nghiệm tại Bảng 4.2.

Bảng 4.2. Ma trận thực nghiệm đánh giá chéo (Cross-Evaluation) trên 250 nhân vật

Train	Test	Kendall Tau	Spearman	EM (%)	Short	Long	Ratio
AT	AT	0.9079	0.9577	32.40%	96.21%	94.50%	0.98
	MT-1	0.7968	0.8808	6.00%	88.51%	91.31%	1.03
	MT-full	0.3919	0.5010	1.20%	71.15%	67.89%	0.95
MT-1	AT	0.8847	0.9474	21.20%	94.57%	93.87%	0.99
	MT-1	0.8358	0.9189	8.40%	91.57%	92.04%	1.01
	MT-full	0.5348	0.6485	2.80%	78.17%	75.16%	0.96
MT-full	AT	0.5701	0.6785	4.00%	60.86%	52.77%	0.86
	MT-1	0.5696	0.6773	4.40%	61.60%	51.85%	0.84
	MT-full	0.5634	0.6712	3.20%	61.20%	51.44%	0.84

4.3.1 Nhận định dựa trên các hệ số đo lường

1. **Hiện tượng học lối tắt (Shortcut Learning):** Khi kiểm thử chéo, **Model AT** thể hiện sự phụ thuộc tuyệt đối vào các con số tường minh. Khi mất mốc thời gian (Test MT-full), chỉ số Tau lao dốc từ 0.9079 xuống 0.3919. Mức Spearman 0.5010 cho thấy dù mô hình vẫn nhận diện được ”chiều hướng” cơ bản, nhưng sự liên kết nhân quả sâu đã bị phá vỡ hoàn toàn (EM chỉ còn 1.2%).
2. **Sự cân bằng tối ưu của chiến lược MT-1: Model MT-1** (che 1 sự kiện lúc huấn luyện) chính là phiên bản hoàn thiện nhất. Nó duy trì tỷ lệ Exact Match trên tập AT rất cao (21.20%), đồng thời chống chịu xuất sắc trên tập MT-full (Tau đạt 0.5348, EM 2.8%). Quan trọng nhất, hệ số ổn định **Ratio L/S** của Model MT-1 liên tục dao động từ 0.96 đến 1.01 trên mọi kịch bản test. Điều này khẳng định kỹ thuật Masking một phần đã ép mô hình học được các định lý nhân quả thực sự, giúp nó giải quyết tốt cả các chuỗi dài phức tạp.
3. **Sự sụp đổ logic khi loại bỏ hoàn toàn dữ kiện thời gian (MT-full): Model MT-full** gặp thất bại trong việc thiết lập không gian vector. Dấu hiệu rõ nhất không nằm ở chỉ số Tau (quanh mức 0.56 – 0.57) mà nằm ở **Ratio L/S** tụt mạnh xuống mức 0.84 – 0.86. Khi không có điểm neo thời gian lúc huấn luyện, mô hình hoàn toàn mất phương hướng

khi số lượng câu tăng lên; chuỗi càng dài (Long Acc $\sim 51\%$), độ chính xác càng giảm mạnh.

4.4 So sánh đối chiếu với các mô hình ngôn ngữ lớn (LLMs)

Nghiên cứu tiến hành đánh giá đối chiếu với hai mô hình LLMs tiên tiến: **DeepSeek-R1-Distill-Qwen-7B** và **Qwen2.5-14B-Instruct** (Zero-shot Prompting). Bảng 4.3 so sánh toàn diện các tham số giữa LLMs và đại diện tối ưu nhất của PhoBERT (Model MT-1).

Bảng 4.3. So sánh đa chiều hiệu suất sắp xếp sự kiện giữa PhoBERT Pairwise và LLMs

Mô hình	Task	Tau	Spearman	EM (%)	Short	Long	Ratio
PhoBERT (MT-1)	AT	0.8847	0.9474	21.20%	94.57%	93.87%	0.99
DeepSeek-7B	AT	0.6470	0.6754	26.96%	94.66%	76.96%	0.81
Qwen2.5-14B	AT	0.5580	0.6299	11.30%	83.62%	75.18%	0.90
PhoBERT (MT-1)	MT-1	0.8358	0.9189	8.40%	91.57%	92.04%	1.01
DeepSeek-7B	MT-1	0.2988	0.3262	5.22%	74.61%	60.71%	0.81
PhoBERT (MT-1)	MT-full	0.5348	0.6485	2.80%	78.17%	75.16%	0.96
DeepSeek-7B	MT-full	0.2388	0.2664	2.17%	73.09%	57.06%	0.78
Qwen2.5-14B	MT-full	0.3842	0.4120	5.20%	75.00%	68.50%	0.91

4.4.1 Đánh giá năng lực sinh ngôn ngữ so với xếp hạng cặp

- **Cạm bẫy ”Cửa sổ ngữ cảnh” của LLMs:** Mặc dù DeepSeek-7B đạt tỷ lệ Exact Match cực cao (26.96%) và Short Acc ngang ngửa PhoBERT (94.66%) trên tập AT, chỉ số **Ratio L/S** của DeepSeek lại ở mức thảm họa (**0.81** trên AT và rớt xuống **0.78** trên MT-full). Điều này bóc trần một giới hạn cố hữu của LLMs: khi yêu cầu sinh ra hoán vị cho chuỗi văn bản dài, mô hình tạo sinh bắt đầu bị ”ảo giác” (hallucination) và đánh mất luồng chú ý (attention) ở các sự kiện cuối.
- **Sức bền định tính của Qwen2.5-14B:** Với lượng tham số gấp đôi (14B), Qwen2.5 không theo đuổi việc trích xuất số liệu sắc bén như DeepSeek mà thể hiện khả năng giữ nhịp ngữ cảnh tốt hơn. Điều này minh chứng qua **Ratio L/S** đạt mức 0.90 – 0.91 trên cả AT

và MT-full. Nhờ ổn định ở các chuỗi dài, Qwen2.5 đạt mức Exact Match cao nhất nhóm LLMs ở bài toán khó MT-full (5.20%).

- **Sự vượt trội của cấu trúc ”Chia để trị” (PhoBERT + Borda):** Bằng cách giải phẫu bài toán thành hàng loạt các câu hỏi nhị phân (Pairwise) thay vì sinh chuỗi một lần, PhoBERT giải quyết triệt để điểm yếu ngữ cảnh dài. Hệ số **Ratio L/S luôn tiệm cận 1.0** ở mọi thiết lập khẳng định rằng phương pháp học biểu diễn thứ hạng (Margin Ranking) ưu việt và ổn định hơn hẳn so với Prompting dựa trên dự đoán Token kế tiếp của LLMs đa năng trong bài toán TEO.

4.5 Tổng kết chương

Chương 4 đã triển khai một hệ thống 6 chiều đo lường toàn diện để bóc tách hành vi suy luận của các hệ thống AI. Thông qua ma trận đánh giá chéo 9 kịch bản trên tập 250 nhân vật, nghiên cứu chứng minh kỹ thuật Masking một phần (MT-1) mang lại hiệu ứng điều chuẩn xuất sắc, ngăn chặn mô hình học vẹt.

Đặc biệt, việc sử dụng các chỉ số phân tách độ dài chuỗi (Short Acc, Long Acc, Ratio L/S) đã làm sáng tỏ giới hạn của các mô hình ngôn ngữ lớn (DeepSeek, Qwen2.5) khi đối mặt với dữ liệu dài. Kiến trúc PhoBERT Pairwise kết hợp Borda Count không chỉ đứng đầu về độ chính xác tương quan thứ hạng (Kendall Tau) mà còn duy trì sự ổn định cấu trúc hoàn hảo (Ratio ≈ 1.0), xác lập một phương pháp tiếp cận mạnh mẽ và tối ưu nhất cho bài toán sắp xếp chuỗi sự kiện lịch sử tiếng Việt.

KẾT LUẬN

Thành tích đã đạt được

Nghiên cứu này đã đạt được những đóng góp cụ thể trên cả hai phương diện xây dựng tài nguyên và phát triển mô hình:

1. **Bộ dữ liệu TEO-TM tiếng Việt đầu tiên:** Hơn 300.000 sự kiện từ hơn 20.000 nhân vật, được thu thập và lọc tự động qua pipeline 5 giai đoạn với nhiều lớp đảm bảo chất lượng.
2. **Pipeline xây dựng dữ liệu tái sử dụng:** Hệ thống module hóa gồm 5 file Python độc lập, có thể mở rộng cho các ngôn ngữ khác hoặc lĩnh vực khác ngoài tiểu sử nhân vật.
3. **Benchmark toàn diện:** So sánh bốn nhóm phương pháp (học máy cổ điển, mBERT, PhoBERT, LLM prompting) trên cùng điều kiện thực nghiệm, cung cấp cơ sở tham chiếu cho nghiên cứu tương lai về TEO-TM tiếng Việt.
4. **Kết quả nền tảng:** PhoBERT-base đạt Accuracy 78.5% và F1 0.781 trên tập kiểm tra – điểm chuẩn (baseline) tốt cho cộng đồng NLP tiếng Việt.
5. **Phân tích lỗi định tính:** Xác định bốn nhóm lỗi chính (sự kiện đồng thời, thiếu kiến thức lịch sử, câu phức tạp, nhân vật ít thông tin) giúp định hướng cải tiến cụ thể.

Hướng phát triển tương lai

1. **Mở rộng và làm giàu bộ dữ liệu:** Thu thập thêm từ báo điện tử, sách lịch sử; bổ sung chú thích quan hệ đa dạng (SIMULTANEOUS, INCLUDES) theo chuẩn TimeML; kiểm tra thủ công 5% mẫu ngẫu nhiên để đánh giá chất lượng.
2. **Tích hợp kiến thức nền:** Khai thác Wikidata và DBpedia tiếng Việt để bổ sung thông tin ngữ nghĩa, giải quyết trực tiếp lỗi loại 2 (thiếu kiến thức lịch sử).
3. **Kiến trúc mô hình nâng cao:** Thử nghiệm Graph Neural Network (GNN) mô hình hóa quan hệ nhiều chiều giữa sự kiện; mô hình lai kết hợp rule-based và neural; PhoBERT-large với thêm tầng attention chuyên biệt.

4. **Ứng dụng downstream:** Áp dụng vào hỏi đáp thời gian (temporal QA), xây dựng timeline tự động, và tóm tắt tiểu sử nhân vật.
5. **Phát hành mã nguồn mở:** Công bố bộ dữ liệu và mã nguồn lên Hugging Face Hub và GitHub để cộng đồng nghiên cứu NLP tiếng Việt có thể tiếp cận và phát triển tiếp.

CHƯƠNG A

CẤU TRÚC THƯ MỤC DỰ ÁN

```
1 NCKH/
2 |-- data/
3 |   |-- final_cleaned_events.csv # Mau du lieu nho (demo)
4 |   |-- raw_events.csv          # Su kien tho truoc loc
5 |   |-- urls.csv                # Danh sach URL mau
6 |-- data_downloading.py         # Giai doan 1: Crawl URL Wikipedia
7 |-- data_processing.py          # Giai doan 2: Xu ly URL -> CSV
8 |-- data_extracting.py          # Giai doan 3: Trich xuất sự kiện
9 |-- data_filtering.py           # Giai doan 4: Lọc & làm sạch
10 |-- data_merging.py             # Giai doan 5: Gộp 3 batch
11 |-- data_cleaned_1.csv          # Batch 1 sau lọc
12 |-- data_cleaned_2.csv          # Batch 2 sau lọc
13 |-- data_cleaned_3.csv          # Batch 3 sau lọc
14 |-- data_cleaned_final.csv      # Du lieu cuoi cung
15 |-- wiki_people_urls.txt        # Danh sach URL raw
16 |-- wiki_people_urls.csv        # Danh sach URL da xu ly
17 |-- wikidata_cache.json         # Cache ket qua Wikidata API
```

Listing A.1. Cấu trúc thư mục dự án NCKH

CHƯƠNG B

DANH SÁCH DANH MỤC WIKIPEDIA SỬ DỤNG

Bảng B.1. Các danh mục Wikipedia tiếng Việt làm seed crawl

Tên danh mục	Nhóm	Mục tiêu
Thể_loại:Nguyên_thủ_quốc_gia	Nổi tiếng	20.000 URL
Thể_loại:Chính_khách	Nổi tiếng	
Thể_loại:Nhà_khoa_học	Nổi tiếng	
Thể_loại:Diễn_viên	Nổi tiếng	
Thể_loại:Ca_sĩ	Nổi tiếng	
Thể_loại:Cầu_thủ_bóng_đá	Nổi tiếng	
Thể_loại:Doanh_nhân	Nổi tiếng	
Thể_loại:Nhà_văn	Nổi tiếng	
Thể_loại:Đạo_diễn	Nổi tiếng	
Thể_loại:Nhân_vật_lịch_sử	Nổi tiếng	
Thể_loại:Nhà_côn_trùng_học	Ít biết	2.000 URL
Thể_loại:Nhà_khí_tượng_học	Ít biết	
Thể_loại:Nhà_thực_vật_học	Ít biết	
Thể_loại:Nhà_hải_dương_học	Ít biết	
Thể_loại:Nhà_ngôn_ngữ_học	Ít biết	
Thể_loại:Nhà_thư_viện_học	Ít biết	
Thể_loại:Nhà_cổ_sinh_vật_học	Ít biết	
Thể_loại:Nhà_thần_học	Ít biết	

CHƯƠNG C

MÔI TRƯỜNG THỰC NGHIỆM

Bảng C.1. Cấu hình phần cứng và phần mềm thực nghiệm

Thành phần	Giá trị
Ngôn ngữ lập trình	Python 3.10
Framework học sâu	PyTorch 2.0, Transformers 4.30
GPU	NVIDIA RTX 3090 (24GB VRAM)
RAM	32GB
Hệ điều hành	Ubuntu 22.04 LTS
Thư viện NLP	underthesea 6.8.0
Thư viện dữ liệu	pandas 2.0, scikit-learn 1.3
Mô hình pretrained	vinai/phobert-base

Tài liệu tham khảo

- [1] James Pustejovsky et al. TimeML: Robust specification of event and temporal expressions in text. In *IWCS-5*, pages 1–9, 2003.
- [2] Marc Verhagen et al. SemEval-2010 task 13: TempEval-2. In *Proceedings of SemEval*, pages 57–62, 2010.
- [3] Qiang Ning, Hao Wu, and Dan Roth. A multi-axis annotation scheme for event temporal relations. In *Proceedings of ACL*, pages 1318–1328, 2018.
- [4] Nathanael Chambers and Dan Jurafsky. Classifying temporal relations between events. In *Proceedings of ACL*, pages 173–176, 2007.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [6] Ben Zhou et al. Going on a vacation takes longer than going for a walk: A study of temporal commonsense understanding. In *Proceedings of EMNLP*, pages 3363–3369, 2019.
- [7] Dat Quoc Nguyen and Anh Tuan Nguyen. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of EMNLP*, pages 1037–1042, 2020.